

Community Hub

Trabajo Práctico 2: CMS y Frameworks Web

Asignatura: Frameworks e Interoperabilidad

Proyecto: Community Hub

Repositorio: <https://github.com/valentin-bustamante/community-hub>

Fecha de preparación: 4 de octubre de 2026

1. Dominio y alcance

Community Hub es un prototipo académico relacionado con el Trabajo Final de una aplicación de comunicación organizada por comunidades, canales y usuarios. El TP demuestra el uso de un framework web y un CMS mediante funciones persistidas y controles de acceso. No pretende implementar todas las capacidades de Discord.

La solución incluye registro e inicio de sesión, creación de comunidades, invitaciones, canales de texto, envío de mensajes y gestión de membresías. No incorpora servicios de terceros, bots, voz, videollamadas ni interoperabilidad con otros sistemas. Next.js y Strapi son los dos componentes de esta misma solución.

2. Elección de las herramientas

Framework: Next.js

Se eligió Next.js porque permite continuar con React y TypeScript, organizar las rutas por archivos y compilar la aplicación con una configuración integrada. App Router reúne páginas, layouts y componentes. La interfaz del chat usa componentes de cliente porque necesita eventos, estado de selección y acceso a la sesión del navegador. En este TP no se atribuyen al proyecto ventajas de renderizado del chat en el servidor que no se hayan implementado.

CMS: Strapi

Strapi permite definir modelos y relaciones, administrar registros desde Content Manager y extender la API con controladores TypeScript. Users & Permissions aporta autenticación por JWT y permisos de acciones. Las reglas específicas por comunidad se implementaron adicionalmente en los controladores. El rol administrador del CMS es independiente del rol propietario de una comunidad.

Alternativas evaluadas y descartadas

Alternativa	Capacidad relevante	Motivo de descarte para este TP
WordPress	Gestión de contenido y REST API	El equipo priorizó modelar entidades de comunidad y escribir autorización en TypeScript. Un modelo centrado en contenido editorial exigiría adaptar tipos y permisos para este dominio.
Directus	Modelo relacional, administración y permisos	Es una alternativa válida. Se priorizó la extensión de controladores y la integración con el ecosistema de Strapi elegido por el equipo.
React con Vite	Interfaz React y servidor de desarrollo	También resuelve el frontend. Se prefirió Next.js para mantener rutas y layout dentro del mismo framework y aprovechar la estructura ya integrada con el template.

Estos motivos describen la adecuación al trabajo y las decisiones del equipo, no una inferioridad general de las alternativas.

3. Arquitectura e implementación

El monorepo usa npm workspaces: apps/frontend contiene Next.js y apps/backend contiene Strapi. El navegador se comunica con la API REST local mediante un cliente HTTP común. Este agrega el JWT, desactiva caché de consultas privadas y convierte errores en mensajes visibles. Una respuesta 401 elimina la sesión y vuelve al login.

SQLite evita instalar un servidor de base de datos adicional para la demostración. La base local, los secretos y las compilaciones quedan fuera de Git. `npm run setup` genera secretos aleatorios sin sobrescribir un entorno existente. El archivo de bloqueo incluye las dependencias opcionales necesarias para diferentes plataformas.

Modelo y relaciones

Entidad	Atributos principales	Relaciones
Comunidad	nombre, código de invitación privado, icono opcional	Una comunidad tiene muchos canales y membresías.
Canal	nombre, documentId	Un canal pertenece a una comunidad y contiene muchos mensajes.
Membresía	rol, documentId	Une un usuario con una comunidad.
Mensaje	contenido, createdAt, documentId	Pertenece a un canal y a un usuario.
Usuario	username, email, identidad del plugin	Tiene membresías y puede enviar mensajes.

El cliente usa `documentId` como identificador estable de Strapi 5. La creación de una comunidad genera el canal general y la membresía de propietario dentro de una transacción. La unión rechaza una membresía existente. El borrado de canales elimina sus mensajes y conserva al menos un canal por comunidad.

4. Módulos desarrollados

Comunidades

La vista consulta las membresías propias, lista comunidades y permite crear una o unirse mediante invitación. La selección define el contexto de toda la pantalla. Solo los miembros pueden consultar el detalle y el código de invitación. El listado REST también limita sus resultados a las comunidades del usuario.

Canales

La barra lateral consulta canales de la comunidad activa. La selección actualiza el encabezado y el chat. Un cambio de comunidad descarta la selección anterior y las respuestas tardías. El propietario puede crear, renombrar y eliminar canales con validación de nombre y prevención de duplicados.

Membresías

El panel muestra usuarios agrupados como propietarios, administradores y miembros. El propietario puede dar o quitar el rol de administrador y expulsar miembros. El backend valida cada acción y bloquea el acceso de una cuenta cuya membresía se revocó. Transferir propiedad y abandonar una comunidad están disponibles en la API, sin controles dedicados en esta interfaz.

Mensajes y autenticación

El chat carga los últimos 50 mensajes y consulta novedades cada tres segundos. Acepta texto de hasta 3000 caracteres, conserva el autor de la sesión y muestra estado vacío, carga y error. El registro e inicio de sesión usan Users & Permissions. El nombre de usuario y el JWT se almacenan en `localStorage`, con cierre de sesión explícito y limpieza al vencer el token.

Librerías

Tailwind CSS define estilos y adaptación a pantallas. Radix UI aporta diálogos y menús con interacción por teclado, cierre por Escape y gestión de foco. shadcn/ui reúne los componentes de interfaz. Lucide aporta iconos. Concurrently permite iniciar ambos servicios desde la raíz. Las pruebas REST usan el runner nativo de Node.js, sin dependencias adicionales.

5. Template y estilo visual

El punto de partida es chat-template, variante whatsapp-mock, atribuido a Manoj (rayimanoj8) en 21st.dev. El commit d235a95 conserva la integración previa a la adaptación. La documentación de procedencia identifica el enlace y la licencia reportada, junto con sus limitaciones de verificación. Las capturas históricas permiten mostrar el original sin depender de la disponibilidad del proveedor.

Elemento original	Modificación aplicada
Lista de contactos ficticios	Comunidades y canales persistidos en Strapi
Conversación y usuario de muestra	Mensajes y autor de la sesión real
Acciones de llamadas y estado	Invitaciones, actualización de datos y gestión de miembros
Paneles de chat genéricos	Encabezado de comunidad/canal y panel por roles
Distribución de escritorio	Navegación y miembros en diálogos para teléfonos
Identidad del generador	Título Community Hub, descripción e idioma español

Se conserva una estética sobria, tipografía Geist, tonos neutros, separación por bordes y selección destacada. Los nombres largos se recortan en navegación y los mensajes se ajustan incluso sin espacios. Cada estado tiene un texto propio. El botón Actualizar comunidad vuelve a consultar membresías y canales y recarga los integrantes y el chat, para reflejar ediciones desde el CMS sin cerrar sesión.

6. Organización y documentación de cambios

Git conserva ramas por tarea, PRs y commits de integración. T-14 concentra la vista principal y T-16 la validación. La consolidación final reúne las correcciones, la limpieza de recursos de ejemplo del blog de Strapi y la documentación. El tablero enlazado desde README permite seguir las tareas. Los docentes deben tener acceso al repositorio y al tablero de acuerdo con la consigna.

7. Validación

La evidencia y los resultados efectivos están en T-16-pruebas-funcionales.md. npm run test:api ejecuta el recorrido sobre un CMS local con cuentas descartables. La validación incluye autenticación, dos comunidades, invitaciones, permisos, canales, mensajes y revocación de acceso. El lint completo y la compilación verifican el código integrado. La revisión visual cubre escritorio y pantallas pequeñas, además de estados de error y navegación por teclado.

8. Límites y mejoras posibles

El historial inicial se limita a 50 mensajes y no tiene paginación de mensajes antiguos. El polling no garantiza una experiencia de chat de alta concurrencia y puede omitir novedades cuando se supera el límite entre consultas. Las imágenes opcionales de comunidad no se renderizan: la interfaz usa iniciales. El backend permite transferir propiedad y abandonar comunidades, pero estos recorridos requieren la API. La sesión en localStorage es adecuada para el prototipo y exige evaluar cookies HttpOnly antes de un despliegue de producción.

La administración directa del CMS requiere respetar las relaciones de dominio. Los controladores de la aplicación aplican reglas de negocio, pero una edición administrativa no equivale a ejecutar esas acciones. No se implementaron correo de invitación, recuperación de contraseña, moderación avanzada ni despliegue productivo.

9. Conclusiones

La separación entre interfaz y CMS permitió reemplazar el chat estático por módulos con datos persistidos. La integración evidencia que los permisos de una acción del CMS deben complementarse con autorización por comunidad. La revisión final corrigió rutas de lectura que podían exponer datos ajenos y mejoró la actualización de contexto y la navegación móvil.

El TP vincula herramientas, modelado, diseño y pruebas con el dominio del Trabajo Final, manteniendo un alcance demostrable para la exposición. Las funciones y las limitaciones quedan documentadas para que los resultados puedan reproducirse y evaluarse.

10. Bibliografía

Documentación oficial consultada el 4 de octubre de 2026.

1. Next.js. App Router: <https://nextjs.org/docs/app>
2. Next.js. Server and Client Components: <https://nextjs.org/docs/app/getting-started/server-and-client-components>
3. Strapi. Content Manager: <https://docs.strapi.io/cms/features/content-manager>
4. Strapi. Users & Permissions: <https://docs.strapi.io/cms/features/users-permissions>
5. Strapi. Customization: <https://docs.strapi.io/cms/customization>
6. Directus. Data Model: <https://docs.directus.io/app/data-model>
7. WordPress. REST API: <https://developer.wordpress.org/rest-api/reference/>
8. 21st.dev. Referencia seleccionada: <https://21st.dev/@rayimanoj8/components/chat-template/whatsapp-mock>
9. Consigna de cátedra. FI_TP2_2026, Trabajo Práctico 2: CMS y Frameworks Web.